Universidade do Minho

Ano Letivo: 2026/27

Sistemas Conexionistas para Aprendizagem Automática

PL3 – Redes Neuronais no PyTorch



Licenciatura em Ciência de Dados, 1º Semestre, 3º Ano

Sumário

1 Perceptron e MLP

2 Pesos, Bias e Ativação

3 Logits e Probs

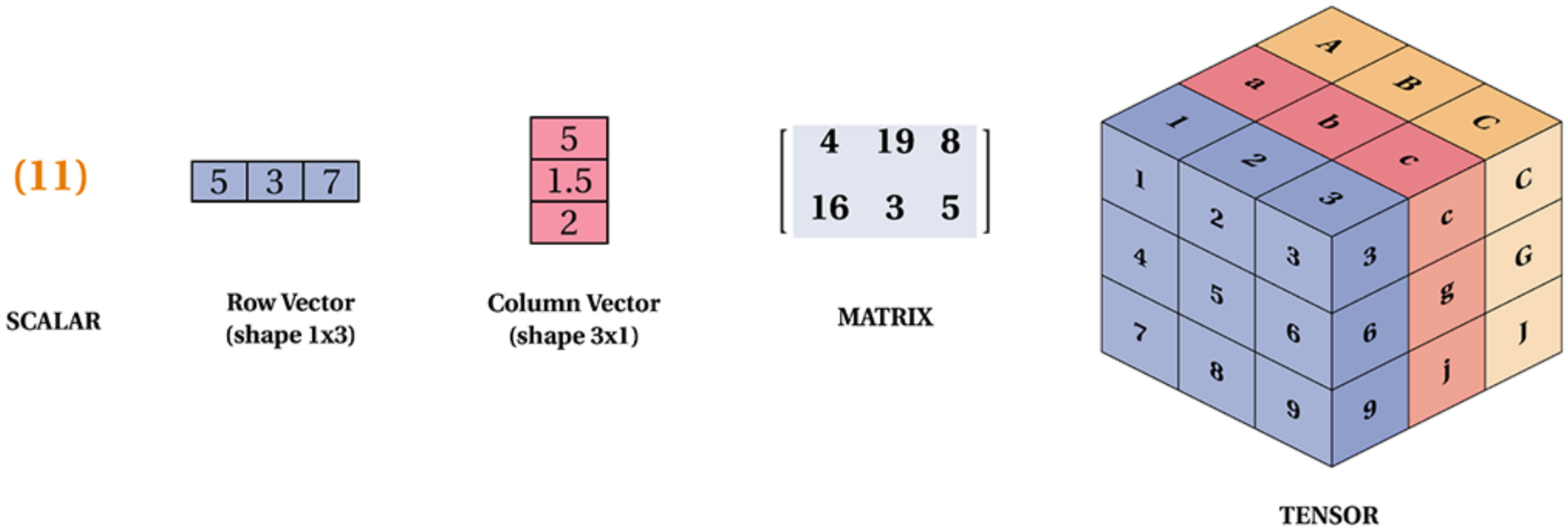
4 Loss e Otimizadores

5 Epochs e Batches

6 Rede Neuronal e Métricas

Recapitulação

- Os **tensores** são a **estrutura de dados fundamental** do PyTorch.
- Um tensor é uma **matriz n-dimensional** que pode ter **nenhuma** dimensão (escalar), **uma** dimensão (vetor), **duas** dimensões (matriz) ou **mais** dimensões.



- Os tensores podem ter operações aplicadas sobre eles, como a adição, a subtração, o produto escalar, o produto vetorial, etc.

Recapitulação

Autograd

- Os tensores com **requires_grad=True** representam os **pesos** e **parâmetros** que **mudam durante o treino**.
- O autograd regista as operações e **loss.backward()** calcula os gradientes.

ATUALIZAÇÃO:

```
#descida do gradiente
with torch.no_grad():
    w -= lr * w.grad

#limpar os gradientes
w.grad.zero_()
```

EXEMPLO:

```
import torch

w = torch.tensor(2.0, requires_grad=True)
loss = (w * 3.0 + 1.0 - 10) ** 2
loss.backward()
print("grad:", w.grad)
```

OUTPUT:

```
grad: tensor(-18.)
```

⚠ No treino não fazemos esta atualização à mão:
o **otimizador** (torch.optim) atualiza os pesos e limpa os gradientes por nós!

PyTorch

Perceptron

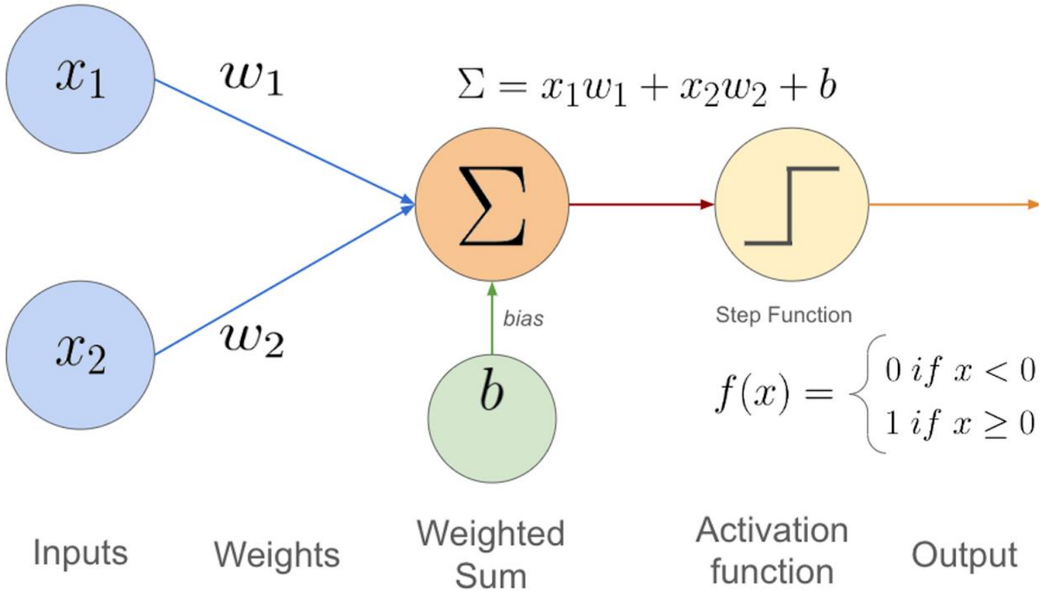
O que é?

- Uma **unidade lógica** de uma rede que mimica um **neurónio** num cérebro humano – inspiração biológica;
- Um algoritmo que recebe **várias** entradas, x1, x2, … e produz um único **output binário** (uma de duas possibilidades);
- Um Perceptron segue o modelo "**feed-forward**", o que significa que as entradas são enviadas para o neurónio, processadas e resultam numa saída.
- **Apenas** é capaz de aprender problemas **linearmente separáveis**.

função de ativação input

$$y=f(w\cdot x+b)$$

output peso bias



Perceptron

Exemplo de Perceptron com PyTorch

```
import torch
from torch import nn
torch.manual_seed(0)

# Dados simples: porta lógica AND
X = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y = torch.tensor([[0.],[0.],[0.],[1.]])

# Modelo Perceptron (sem camadas ocultas): 2 entradas -> 1 logit
model = nn.Linear(2, 1)
loss_fn = nn.BCEWithLogitsLoss()     # aplica a sigmoid internamente
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)

for epoch in range(1000):
    loss = loss_fn(model(X), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

print(torch.sigmoid(model(X)))       # probabilidades
```

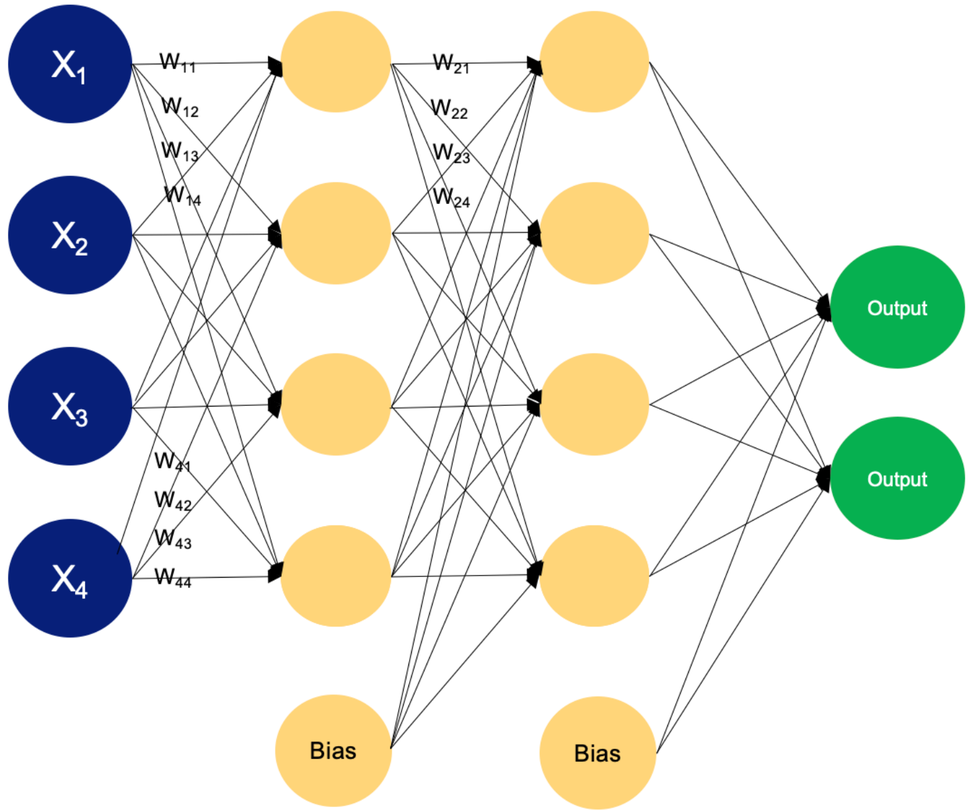
OUTPUT (≈):

```
[[0.00],
 [0.04],
 [0.04],
 [0.94]]
```

Multi-Layer Perceptron (MLP)

O que é?

- É uma **extensão** do perceptron que contém **pelo menos uma camada oculta**, sendo que cada camada contém **qualquer número de nós**.
- Permite aprender **relações não lineares;**
- O objetivo de um MLP é modelar **relações complexas** entre entradas e saídas.
- Cada nó de uma camada liga-se a cada nó da camada seguinte.



Multi-Layer Perceptron (MLP)

Exemplo de MLP com PyTorch

```
import torch
from torch import nn
torch.manual_seed(0)

# Dados de entrada (XOR)
X = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y = torch.tensor([[0.],[1.],[1.],[0.]])

# MLP com uma camada oculta
model = nn.Sequential(
    nn.Linear(2, 4), nn.ReLU(),      # camada oculta
    nn.Linear(4, 1)                  # saída (1 logit)
)
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.05)

for epoch in range(500):
    loss = loss_fn(model(X), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

print(torch.sigmoid(model(X)))
```

OUTPUT (≈):

```
[[0.00],
 [1.00],
 [1.00],
 [0.00]]
```

Pesos e Vieses

Pesos/Weights

- Os pesos são **valores numéricos** que determinam a **influência** de **cada entrada** na **saída final** da rede.
- Pesos positivos reforçam a entrada. Pesos negativos inibem a entrada.
- Se um peso for **0**, então a sua característica correspondente **não contribui** para o modelo.
- **Durante o treino**, os pesos são **atualizados** através de **algoritmos de otimização** para minimizar a diferença entre os resultados previstos e os reais.
- Os pesos são os coeficientes da equação que estamos a tentar resolver!

Exemplo: Numa rede neural que prevê os preços dos imóveis, o peso para o "tamanho da casa" determina o quanto o tamanho da casa influencia a previsão do preço. Quanto maior for o peso, maior será o impacto que o tamanho terá no resultado final.

Pesos e Vieses

Bias/Vieses

- Ao contrário dos pesos, não estão ligados a nenhuma entrada específica, mas, em vez disso, **deslocam** a **função de ativação** para melhor se ajustar aos dados.
- Os **bias** ajudam os **neurónios a ativarem-se** mesmo quando a soma ponderada das entradas não é suficiente.
- Sem bias, os neurónios só seriam ativados quando a entrada atingisse um limiar específico. Isto torna a rede mais flexível, permitindo a ativação numa gama mais ampla de condições.
- **Durante o treino**, os bias são **atualizados** juntamente com os pesos através de **backpropagration**. Juntos, ajustam o modelo, melhorando a precisão da previsão.

Exemplo: Numa rede neural que prevê os preços dos imóveis, o enviesamento (bias) garante que, mesmo para uma casa com área zero, o modelo prevê um preço base diferente de zero. Esse valor pode refletir o valor do terreno, custos administrativos, etc.

RESUMINDO:

Weights → controlam a importância de cada entrada
Bias → desloca a função de ativação (permite ajustar o limiar de ativação)

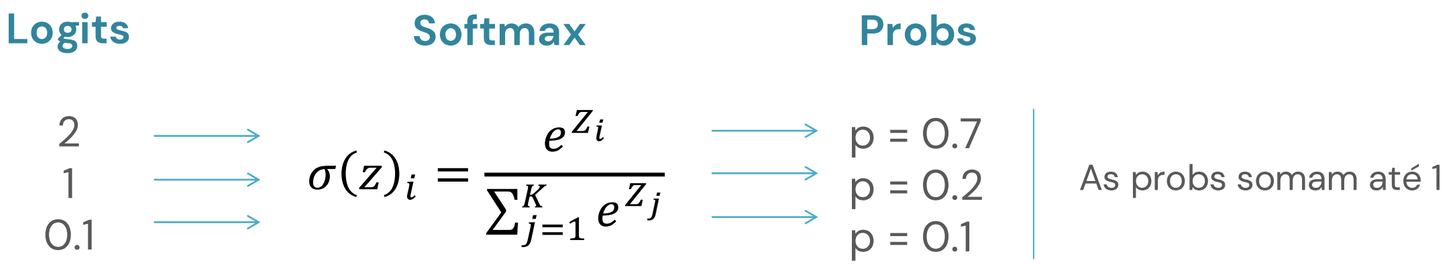
Funções de Ativação

- Uma função de ativação $f(z)$ define **como o neurónio responde** à soma ponderada das entradas.
- Sem ela, a rede seria apenas uma **regressão linear**.
- As funções de ativação introduzem a **não linearidade** no modelo.

Função	Fórmula	Descrição	Quando usar	Representação Gráfica
Step (Degrau)	$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$	Ativa o neurônio apenas quando o valor de entrada ultrapassa um limiar. Produz saídas binárias (0 ou 1).	Usada no **Perceptron original** (modelo histórico). Não é usada em redes modernas porque não é derivável.	
Sigmoid (Logística)	$f(x) = \frac{1}{1+e^{-x}}$	Suaviza a saída para o intervalo (0, 1); transforma valores em probabilidades.	**Camada de saída** em classificadores binários. Boa quando queremos interpretar o resultado como probabilidade.	
Tanh (Tangente hiperbólica)	$f(x) = \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	Saída entre (-1, 1), centrada em zero. Similar à sigmoid, mas com valores negativos.	**Camadas ocultas** quando os dados são normalizados em torno de 0.	
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$	Retorna 0 para valores negativos e x para positivos. Simples, rápida e evita saturação.	**Camadas ocultas** em redes profundas (CNNs, MLPs). É a ativação mais usada atualmente.	
Softmax	$f_i(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	Converte um vetor de saídas em probabilidades que somam 1.	**Camada de saída** em problemas de **classificação multiclasse**.	

Logits e Probabilidades em Redes Neurais

- **Logits** são os **valores brutos** que saem da última camada da rede, **antes da função de ativação**.
- Representam a **força relativa da evidência** de cada classe, mas **não estão normalizados** (pode variar de [-∞, +∞]).
- Os **logits** tornam-se normalmente uma **entrada** para a **função *softmax* (multiclasse) ou *sigmoid* (binária)**.
- A **função de ativação** gera então um **vetor de probabilidades – probs -** (**normalizadas**) com um valor para cada classe.
- Em PyTorch, a **CrossEntropyLoss** já aplica a **softmax**: o modelo devolve **logits**.
- As **Probabilidades** são os valores **normalizados** entre 0 e 1, indicando o **grau de confiança** da rede em cada classe.



Loss e Gradiente

O que é a Função de Perda (Loss)?

- A **função de perda** mede **o quão errada está a previsão** da rede em relação ao valor real.
- Quanto **menor** a perda, **melhor** o modelo.
- O objetivo do treino é **minimizar essa função** ajustando os pesos e bias da rede.
- É utilizada para estimar a perda do modelo para que os pesos possam ser atualizados para reduzir a perda na próxima avaliação.

Loss e Gradiente

Regressão

Previsão de valores contínuos (ex: preço de imóveis, temperatura, etc.). É comum usar Mean Absolute Error (MAE, nn.L1Loss), Mean Squared Error (MSE, nn.MSELoss) ou Rooted Mean Squared Error (RMSE).

$$MAE = \frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j| \qquad MSE = \frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2 \qquad RMSE = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2}$$

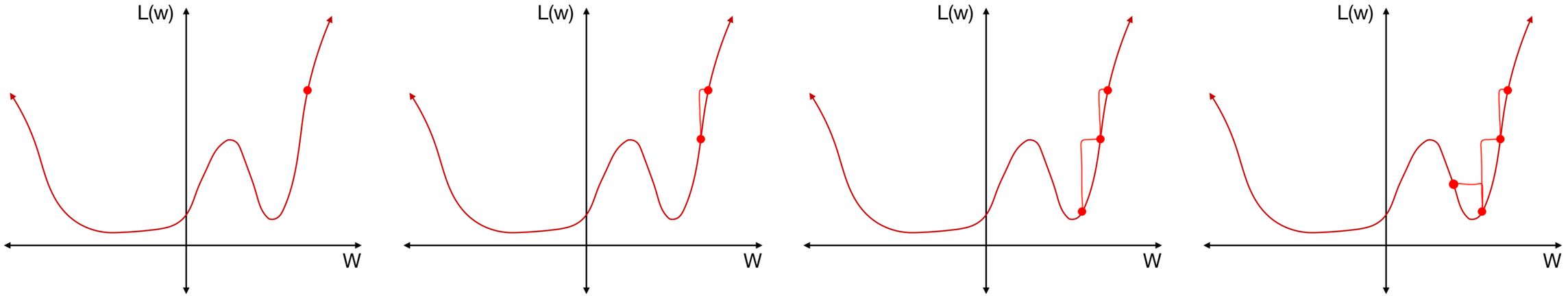
Classificação

Tipo de problema	Função de perda	Fórmula	Quando usar
Classificação binária	Binary Cross-Entropy nn.BCEWithLogitsLoss	$L = -[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$	Classificação 0/1 (ex: fraude, doença, etc.)
Classificação multiclasse (inteiros)	Cross-Entropy nn.CrossEntropyLoss	$L = -\log(\hat{y}_{y_{true}})$	Quando as classes são índices inteiros (ex: 0, 1, 2, 3).
Classificação multiclasse (one-hot)	Cross-Entropy nn.CrossEntropyLoss	$L = -\sum y_i \log(\hat{y}_i)$	Quando os alvos são one-hot ou probabilidades (ex: [0., 0., 1., 0.]).

Loss e Gradiente

Gradient Descent

- É um algoritmo de **otimização iterativa** utilizado para **minimizar** uma **função de custo**, ajustando os parâmetros do modelo na direção da **descida mais acentuada** do gradiente da função.
- O gradiente indica-nos **a direção** na qual **atualizar** os pesos em relação ao valor de perda obtido, garantindo a **convergência** para os **valores ótimos** dos parâmetros.
- Em termos simples, encontra os valores **ótimos dos pesos** e **bias reduzindo gradualmente o erro** entre os resultados previstos e os reais.



$$W_{t+1} = W_t - \frac{\partial L(w)}{\partial W_x} \cdot lr$$

- L(w) define o valor da perda (dependendo dos pesos).
- W define os pesos.
- lr define a learning rate (tamanho dos passos ao descer o gradiente).

Aplicação Prática

Ficha de exercícios 3 – Parte II (h)



Otimizadores e Métricas

Os **otimizadores** controlam **como os pesos são ajustados** durante o treino, usando as informações dos **gradientes** (calculados a partir da função de perda).

No PyTorch, existem muitos otimizadores no módulo torch.optim (o autograd calcula o *backward* a partir do *forward*)!

torch.optim.SGD
torch.optim.Adagrad
torch.optim.SGD(momentum=0.9)
torch.optim.Adam
...



Otimizadores modernos ajustam automaticamente **o tamanho e a direção do passo**.

Epochs e Batch Processing

⚠ Treinar o dataset de uma vez pode ser muito pesado (memória e processamento).

O que é o Batch size?

- Define o **número de amostras de treino** processadas **antes** de os parâmetros internos do modelo (hiperparâmetros) serem **atualizados**.

- Um *batch size* de 32 significa que são utilizadas 32 amostras. No final do lote, as previsões são comparadas com as *labels* esperadas. O erro é então calculado. A partir deste erro, por *backpropagation*, é possível atualizar os pesos do modelo antes de o próximo lote de 32 amostras ser processado.

Batch = conjunto de exemplos processados de uma só vez.
Batch size = número de exemplos por lote.



Como selecionar o batch size?
Batch muito pequeno → ruído maior
Batch muito grande → treino mais lento e pesado

Epochs e Batch Processing

Batch Gradient Descent

Batch Size = Tamanho do Conjunto de Treino
1 passo de de descida de gradiente em 1 epoch
Todos os exemplos para cada passo de Gradient Descent

Stochastic Gradient Descent

Batch Size = 1
N passos de descida do gradiente por epoch (onde N é o tamanho do Conjunto de Treino)
Converge mais rapidamente, mas é computacionalmente mais lento

Mini-Batch Gradient Descent

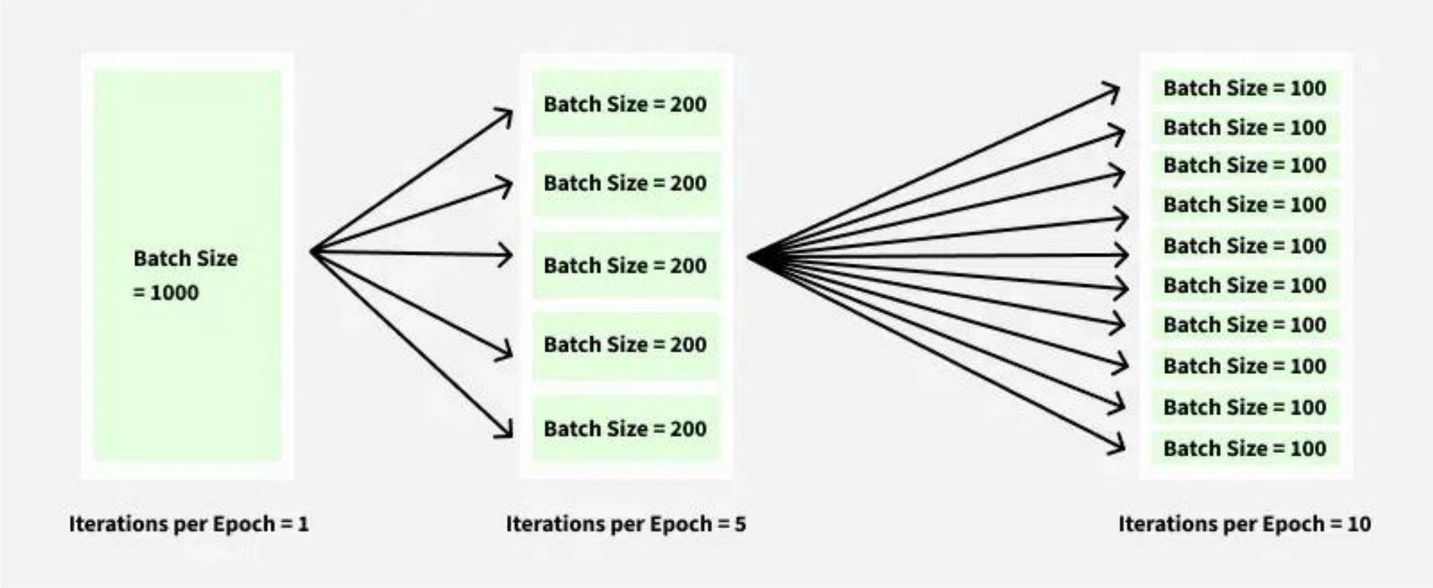
1 < Batch Size < Tamanho do Conjunto de Treino
Um lote com o tamanho 32 significa que vamos dividir todo o conjunto de dados em lotes de 32 elementos. Para um conjunto de dados com 2048 exemplos, teremos 64 lotes (iterações) de 32 elementos cada.

Epochs e Batch Processing

O que é uma Epoch?

Uma **passagem de treino completa** sobre **todo o conjunto de dados**, de modo a que cada exemplo tenha sido visto uma vez. Por exemplo, se tiver 1000 exemplos e treinar por 10 epochs, cada exemplo será usado 10 vezes para atualizar os pesos.

$$\text{Iterations per epoch} = \frac{\text{Número de exemplos}}{\text{Batch size}}$$



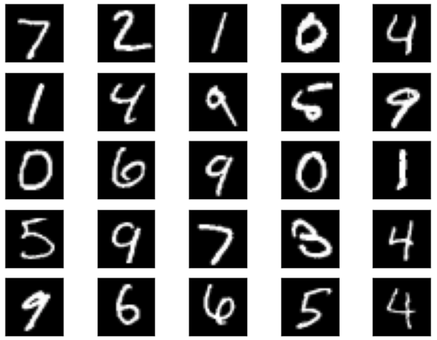
⚠️ Mais epochs → maior probabilidade de convergência, mas cuidado com **overfitting**.

Exemplo de uma Rede Neuronal

1. Carregar dados

O dataset MNIST contém imagens 28×28 de dígitos escritos à mão (0–9).

```
train_ds = datasets.MNIST('data', train=True,  download=True, transform=ToTensor())
test_ds  = datasets.MNIST('data', train=False, download=True, transform=ToTensor())
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
test_loader  = DataLoader(test_ds, batch_size=1000)
```



train_ds[i] → (imagem, label)
DataLoader → entrega os dados em lotes (batches)

2. Pré-processamento dos dados

- Cada imagem tem tamanho 28×28 = 784 pixels.
- As redes densas esperam **vetores** (1D), não **matrizes** (2D).
- É necessário **achatar** (nn.Flatten) e **normalizar** os valores de pixel (ToTensor → [0, 1]).

Exemplo de uma Rede Neuronal

3. Criar uma rede sequencial

- ***nn.Sequential*** = modelo em que as camadas são **empilhadas uma após a outra**.
  = utilizado para pilhas de camadas de **entrada única** e **saída única**.

- Modelo **não linear** → **Funções de Ativação** – estas acrescentam/introduzem não linearidade ao nosso modelo.

Função	Descrição	Onde usar
ReLU (nn.ReLU)	Ativa só valores positivos → simples e eficiente	Camadas ocultas
Sigmoid	Produz valores entre 0 e 1	Saída binária
Softmax	Gera probabilidades somando 1	Saída multiclasses (já incluída na CrossEntropyLoss)

```
model = nn.Sequential(
    nn.Flatten(),                  # 28×28 → vetor de 784
    nn.Linear(784, N), nn.ReLU(),  # camada oculta 1 com N neurónios
    ...
    nn.Linear(N, 10)               # saída (10 classes) → logits
)
```

Exemplo de uma Rede Neuronal

3. Criar uma rede sequencial

```
model = nn.Sequential(
    nn.Flatten(),                   # 28×28 → vetor de 784
    nn.Linear(784, N), nn.ReLU(),   # camada oculta 1 com N neurónios
    ...
    nn.Linear(N, 10)                # saída (10 classes) → logits
)
```

- **Quantos neurónios usar?**

  - Normalmente utiliza-se **potências de 2**: 8, 16, 32, 64, 128, 256, 512…
    - Convenção comum em redes neurais.
    - Melhora eficiência em GPUs.
    - Fácil de ajustar e comparar modelos.

  - **Poucos** neurónios → mais **rápido**, menos overfitting, mas pode **não se adaptar a dados complexos**

  - **Muitos** neurónios → mais **lento**, mais capacidade de **aprender padrões complexos**, mas maior **risco** de **overfitting**.

Modelos estruturados

Como estruturar um modelo do PyTorch para ser facilmente reutilizável?



Criar uma classe que herda de nn.Module

```python
class OurModel(nn.Module):
    # Construtor - define as camadas
    def __init__(self, n_hidden=128):
        super().__init__()
        self.flatten = nn.Flatten()
        self.hidden = nn.Linear(784, n_hidden)
        self.out = nn.Linear(n_hidden, 10)

    # Forward - como os dados passam pelas camadas
    def forward(self, x):
        x = self.flatten(x)
        x = torch.relu(self.hidden(x))
        return self.out(x)

model = OurModel()   # a mesma rede
logits = model(x)    # chama forward(x)
```

Exemplo de uma Rede Neuronal

4. Definir a perda e o otimizador

Sem compile(): definimos **como** o modelo vai aprender:

- **Função de perda (loss):** mede o erro.
- **Otimizador:** ajusta os pesos para reduzir o erro.
- **Métricas:** calculadas por nós (ex.: accuracy).

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(
    model.parameters(), lr=1e-3
)
```

`Adam` → otimizador adaptativo (funciona bem na maioria dos casos).
`CrossEntropyLoss` → quando as labels são números inteiros; aplica o softmax internamente (o modelo devolve logits).

5. Treinar o modelo

O treino é um ciclo sobre os dados, definido por:

- **número de *epochs*** = quantas vezes o modelo vê todos os dados.
- tamanho do lote (***batch_size***) = definido no DataLoader.
- o **ciclo de treino**, que em PyTorch é escrito por nós (próximo slide).

Exemplo de uma Rede Neuronal

5. Treinar o modelo

Cada iteração do ciclo repete os mesmos passos:
• **forward** e **perda** → o autograd regista o grafo.
• **backward** e **step** → calcula os gradientes e atualiza os pesos.

```
for epoch in range(5):                  # 5 passagens pelos dados
    model.train()                       # modo de treino
    for x, y in train_loader:           # um lote de cada vez
        logits = model(x)               # 1. forward
        loss = loss_fn(logits, y)       # 2. calcula o erro
        optimizer.zero_grad()           # 3. limpa os gradientes anteriores
        loss.backward()                 # 4. backward: calcula os gradientes
        optimizer.step()                # 5. atualiza os pesos
    print(epoch, loss.item())
```

Exemplo de uma Rede Neuronal

6. Avaliar o modelo

```
model.eval(); correct = 0
with torch.no_grad():
    for x, y in test_loader:
        correct += (model(x).argmax(1) == y).sum().item()
test_acc = correct / len(test_ds)
```

- model.eval() → modo de avaliação (afeta camadas como o dropout).
- torch.no_grad() → não regista operações: mais rápido, menos memória.
- argmax(1) → classe com maior valor (logit) para cada imagem.

7. Gerar previsões

- Depois de treinar e avaliar, usamos o modelo para **prever novos** exemplos.
- O modelo devolve *logits*; ***torch.softmax()*** converte-os em **probabilidades**.

```
predictions = torch.softmax(model(x_test), dim=1)
```

→ x_test, y_test = next(iter(test_loader))
→ retorna um tensor com formato (n_amostras, 10)
→ cada linha tem 10 valores (probabilidades das classes 0–9)

```
predictions = torch.softmax(model(x_test[:12]), dim=1)
```

→ usa indexação de fatia (*slice*), tal como no NumPy, para usar somente as 12 primeiras imagens do lote de teste.
→ retorna um tensor com formato (12, 10)

- ***predictions[i].argmax()*** → seleciona o índice da classe com maior probabilidade.
- Comparar com ***y_test[i]*** → ver se acertou.

Aplicação Prática

Ficha de exercícios 4 – Parte I



Blackboard

Otimizadores e Métricas

Métricas

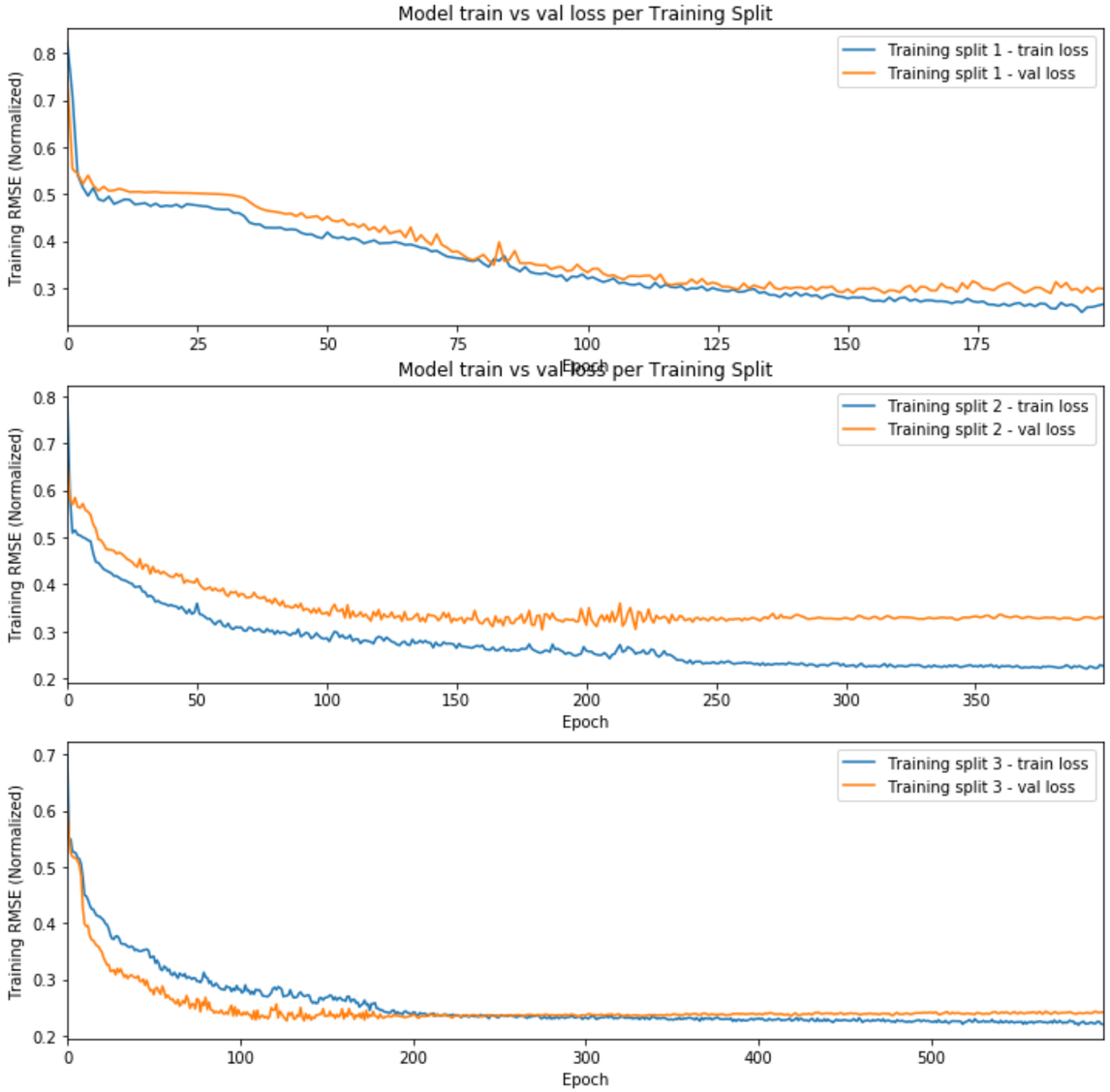
- Uma função utilizada para **avaliar** o **desempenho** do modelo.
- Diferente da *loss*, **não afeta o treino**, apenas o monitoriza. Em PyTorch calcula-se à mão, com o scikit-learn ou com o torchmetrics.

Tipo de problema	Métrica comum	Descrição
Classificação binária	accuracy, precision, recall, AUC	Mede acertos, precisão e sensibilidade.
Classificação multiclasse	accuracy, top-k accuracy	Mede a taxa de acertos ou se a classe correta está entre as K melhores previsões.
Regressão	MAE, MSE, R^2	Mede erro médio ou quadrático entre valores reais e previstos.

Epochs e Batch Processing

Curvas de Aprendizagem

É comum criar gráficos de linhas, chamados **curvas de aprendizagem**, que traçam as epochs (ao longo do eixo x) pelo erro do modelo (no eixo y).



Curvas de Aprendizagem

Em PyTorch

O ciclo de treino não devolve um histórico: guardamos nós a perda de cada epoch.
- **loss.item()** → converte a perda num número Python.
- **média por epoch** → um ponto da curva de aprendizagem.

```
history = []
for epoch in range(5):
    total = 0.0
    for x, y in train_loader:
        loss = loss_fn(model(x), y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total += loss.item()                    # soma a perda de cada lote
    history.append(total / len(train_loader))  # perda média da epoch

plt.plot(history)                               # curva de aprendizagem
plt.xlabel('epoch'); plt.ylabel('loss')
```

Epochs e Batch Processing

Termo	Definição	Efeito no treino
Epoch	Passagem completa de todos os dados	Mais epochs → melhor aprendizagem, mas risco de overfitting
Batch	# exemplos processados por atualização	Tamanho pequeno → mais atualização, ruído alto; Tamanho grande → atualização estável, menos ruído

Fluxo para definir o batch size e epochs

1.º Selecionar os valores iniciais:
Batch size: Comece com 32 ou 64.
Epochs: Comece com 50 ou 100.

2.º Treinar o modelo: Monitorize o desempenho nos conjuntos de treino e validação.

3.º Ajustar com base nas observações:
- Aumente o tamanho do lote se o modelo estiver instável ou o treino for demasiado lento.
- Aumente o número de epochs se o modelo estiver subajustado.

4.º Iterar e refinar

Resumo dos conceitos-chave

Neuron (Perceptron) - Unidade básica de processamento.

Multi-Layer Perceptron (MLP) – Extensão do perceptron, possui camadas ocultas e permite aprender relações não lineares.

Pesos (Weights) - Controlam a importância de cada entrada e são ajustados durante o treino.

Bias - Desloca a função de ativação.

Função de Ativação - Define a saída do neurónio (ReLU, Sigmoid, Softmax, …).

Loss Function - Mede o erro da previsão (MSE, Cross-Entropy, …).

Gradiente (∇) - Indica como ajustar os pesos para reduzir a perda.

Otimização - Processo de ajustar pesos usando o gradiente (Adam, Adagrad, …).

Learning Rate (η) - Tamanho do passo na atualização dos pesos. Valor muito alto → instável; Valor muito baixo → lento.

Batch size - Subconjunto de exemplos processado de cada vez antes da atualização dos pesos.

Epoch - Uma passagem completa por todos os dados de treino, de modo a que cada exemplo tenha sido visto uma vez.

Métrica - Mede o desempenho do modelo (Accuracy, Precision, Recall,..) - não é usada no treino.

Resumo do processo

- Entrada → [Pesos + Bias] → Função de Ativação → Saída
- Cálculo da perda → Gradiente → Atualização dos pesos (via Otimizador)
- Repetir por epochs até a perda estabilizar
- Avaliar desempenho com métricas

💬 *"Treinar uma rede neural é um ciclo contínuo de previsão, erro e correção — até que os pesos capturem o padrão escondido nos dados."*

Aplicação Prática

Ficha de exercícios 4 – Parte II

